

Trevon Woods
Sitaram Ayyagari
A.I. at the Edge - ITAI - 3377
02/07/2026

L03 Reflection Journal

The initial setup for this lab was fairly straightforward and manageable. Since I already had Python and VS Code installed on my computer, the initial preparation didn't take long. I really only needed to install TensorFlow and the Edge Impulse CLI, which was a routine process and completed within a few minutes. Once those were downloaded, I moved on to loading the MNIST dataset. I scaled the pixel values to a range between 0 and 1, which is an important step for helping the model train more effectively. After the data was preprocessed, I began building the model architecture using Keras' Sequential API. The network contained many layers, first was the Conv2D layer with ReLU activation, followed by max pooling, flattening, and two dense layers. The first dense layer processed the extracted features through 128 neurons with another ReLU activation. The final dense layer served as a classification layer that produced the prediction vector for the MNIST classes. All of this was followed by training, then converting and saving the model to a .tflite file to be used on an edge device.

The deployment process was also pretty standard except for the challenge of figuring out another way to upload my model to Edge Impulse since the CLI command for the uploader didn't work. It was a simple fix as I just uploaded the model manually in the project dashboard and requested performance profiling across a range of edge devices. Despite this, the rest of the deployment was fairly smooth. All I had to do was either scan a QR code for use on an edge device or directly build the model and run inference in a separate browser. I also noticed Edge Impulse provides built-in options for training models within the platform. I will definitely be exploring that for future projects.

The most disappointing part of the lab was the testing phase. Even though the model showed high accuracy during training and validation, its real world predictions were consistently incorrect. When I ran inference for the model directly in Edge Impulse I fed it a digit image of a six and it predicted that it was a seven. Then during the live testing of the model on an edge device, I showed it a picture of a seven and it predicted that it was an eight. Interestingly, the model seemed to always predict one class higher for each digit. I am not really sure why this happened but I will be investigating this further since the validation and deployment accuracies were so different.

Overall this lab was a valuable learning experience. It helped me better understand the full workflow of building, converting, deploying, and testing a machine learning model on edge hardware. At the same time, it highlighted how important it is to verify that models behave correctly after deployment, even when training metrics appear strong.

```

● (venv) (base) Deploy_AI_on_Simulated_Edge % pip install tensorflow
Collecting tensorflow
  Downloading tensorflow-2.20.0-cp313-cp313-macosx_12_0_arm64.whl.metadata (4.5 kB)
Collecting absl-py>=1.0.0 (from tensorflow)
  Downloading absl_py-2.4.0-py3-none-any.whl.metadata (3.3 kB)
Collecting astunparse>=1.6.0 (from tensorflow)
  Downloading astunparse-1.6.3-py2.py3-none-any.whl.metadata (4.4 kB)
Collecting flatbuffers>=24.3.25 (from tensorflow)
  Downloading flatbuffers-25.12.19-py2.py3-none-any.whl.metadata (1.0 kB)
Collecting gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 (from tensorflow)
  Downloading gast-0.7.0-py3-none-any.whl.metadata (1.5 kB)
Collecting google_pasta>=0.1.1 (from tensorflow)
  Downloading google_pasta-0.2.0-py3-none-any.whl.metadata (814 bytes)
Collecting libclang>=13.0.0 (from tensorflow)
  Downloading libclang-18.1.1-py2.py3-none-macosx_11_0_arm64.whl.metadata (5.2 kB)
Collecting opt_einsum>=2.3.2 (from tensorflow)
  Downloading opt_einsum-3.4.0-py3-none-any.whl.metadata (6.3 kB)

```

```

● (venv) (base) Deploy_AI_on_Simulated_Edge % npm install -g edge-impulse-cli
npm warn deprecated uuid@3.4.0: Please upgrade to version 7 or higher. Older versions may use Math.random() in certain circumstances, which is known to be problematic. See https://v8.dev/blog/math-random for details.
npm warn deprecated tar@7.4.3: Old versions of tar are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me
npm warn deprecated gauge@2.7.4: This package is no longer supported.
npm warn deprecated are-we-there-yet@1.1.7: This package is no longer supported.
npm warn deprecated npmlog@4.1.2: This package is no longer supported.
npm warn deprecated har-validator@5.1.5: this library is no longer supported
npm warn deprecated glob@10.5.0: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me
npm warn deprecated request@2.88.2: request has been deprecated, see https://github.com/request/request/issues/3142
npm warn deprecated @zeit/dockerignore@0.0.5: "@zeit/dockerignore" is no longer maintained

```

```

on_edge.py > ...
1 import tensorflow as tf
2 from tensorflow.keras.datasets import mnist
3
4 (x_train, y_train), (x_test, y_test) = mnist.load_data()
5 x_train, x_test = x_train / 255.0, x_test / 255.0
6 x_train = x_train.reshape((-1, 28, 28, 1))
7 x_test = x_test.reshape((-1, 28, 28, 1))
8
9 model = tf.keras.Sequential([
10     tf.keras.layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
11     tf.keras.layers.MaxPooling2D((2, 2)),
12     tf.keras.layers.Flatten(),
13     tf.keras.layers.Dense(128, activation='relu'),
14     tf.keras.layers.Dense(10, activation='softmax')
15 ])
16
17 model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
18 model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))
19
20 converter = tf.lite.TFLiteConverter.from_keras_model(model)
21 tflite_model = converter.convert()
22
23 with open('model.tflite', 'wb') as f:
24     f.write(tflite_model)
25

```

```

● (venv) (base) Deploy_AI_on_Simulated_Edge % python3 on_edge.py
/Users/ /Documents/AI_on_Edge/Deploy_AI_on_Simulated_Edge/venv/lib/python3.13/site-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/5
1875/1875 — 8s 4ms/step - accuracy: 0.9541 - loss: 0.1548 - val_accuracy: 0.9803 - val_loss: 0.0605
Epoch 2/5
1875/1875 — 7s 4ms/step - accuracy: 0.9834 - loss: 0.0543 - val_accuracy: 0.9842 - val_loss: 0.0496
Epoch 3/5
1875/1875 — 8s 4ms/step - accuracy: 0.9895 - loss: 0.0343 - val_accuracy: 0.9866 - val_loss: 0.0402
Epoch 4/5
1875/1875 — 7s 4ms/step - accuracy: 0.9922 - loss: 0.0242 - val_accuracy: 0.9881 - val_loss: 0.0406
Epoch 5/5
1875/1875 — 8s 4ms/step - accuracy: 0.9945 - loss: 0.0166 - val_accuracy: 0.9858 - val_loss: 0.0473

```

```
(venv) (base) Deploy_AI_on_Simulated_Edge % edge-impulse-uploader --api-key
model.tflite

Edge Impulse uplo Follow link (cmd + click)
Endpoints:
API: https://studio.edgeimpulse.com
Ingestion: https://ingestion.edgeimpulse.com

Upload configuration:
Label: Not set, will be inferred from file name
Category: training
Cannot handle this file, only .wav, .cbor, .json, .jpg, .jpeg, .png, .csv, .txt, .mp4, .avi, .parquet supported: model
.tflite
```

Upload pretrained model - Step 1: Upload a model

1. Upload your trained model

Upload a TensorFlow SavedModel (.saved_model.zip), ONNX model (.onnx), TensorFlow Lite model (.tflite), LGBM model (.txt), XGBoost model (.json), NGBoost model (ngboost.pkl) or pickle model (.pkl) to get started.

Choose File

model.tflite

2. Model performance

Do you want performance characteristics (latency, RAM and ROM) for a specific device?

☒ No, show me performance for a range of device types.

☐ Yes, run performance profiling for:

Cortex-M4F 80MHz

Upload progress

Cancel

Creating job... OK (ID: 43772256)

✓Job scheduled at 07 Feb 2026 17:39:57

✓Job started at 07 Feb 2026 17:40:00

Processing model...

On-device performance

MCUs

		EON COMPILER		TFLITE	
DEVICE	LATENCY	RAM	ROM	RAM	ROM
Low-end MCU ⓘ	15,269 ms.	107.5K	2.7M	130.9K +23.4K	2.7M +18.9K
High-end MCU ⓘ	198 ms.	107.6K	2.7M	131.0K +23.4K	2.7M +27.3K
+ AI accelerator ⓘ	198 ms.	107.6K	2.7M	131.0K +23.4K	2.7M +27.3K

Microprocessors

DEVICE	LATENCY	MODEL SIZE
CPU ⓘ	8 ms.	2.7M
GPU or accelerator ⓘ	2 ms.	2.7M

Run this model

Scan QR code or launch in browser to test your prototype



Launch in browser

Check model behavior

Upload test data to ensure correct model settings and proper model processing. *(Optional)*

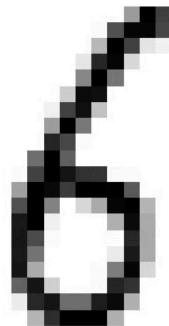
Upload an image

Upload an image to try out your model. The image will be automatically resized to 28x28 (Grayscale).

Choose File 6 Image 25.jpeg

Test sample

Results



INDEX	LABEL	PROBABILITY
0	class 1	0.00
1	class 2	0.00
2	class 3	0.00
3	class 4	0.00
4	class 5	0.00
5	class 6	0.01
6	class 7	0.83
7	class 8	0.00
8	class 9	0.16
9	class 10	0.00

 Modify model settings if results do not reflect expected model behavior, otherwise save and deploy your model!



twoods3693 / mnist_classification

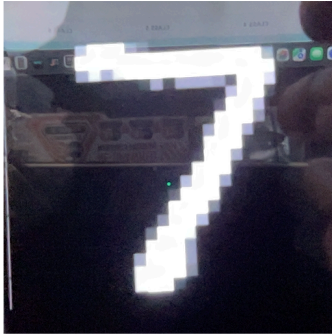


Building project...

es++: warning: running limited binaryen optimizations because DWARF info requested (or indirectly required) [-Wlimited-postlink-optimizations]

Switch to data collection mode

🔗 This client is [open source](#).



Inferencing...

class 8 (0.82)

Time per inference: 2 ms.

CLASS 1 CLASS 2 CLASS 3 CLASS 4 CLASS 5 CLASS 6 CLASS 7 CLASS 8 CLASS 9 CLASS 10

	CLASS 1	CLASS 2	CLASS 3	CLASS 4	CLASS 5	CLASS 6	CLASS 7	CLASS 8	CLASS 9	CLASS 10
3173	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00	0.00	0.00
3172	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00	0.00	0.00
3171	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00	0.00	0.00
3170	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00	0.00	0.00
3169	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00	0.00	0.00
3168	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00	0.00	0.00
3167	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.99	0.00	0.00
3166	0.00	0.00	0.01	0.00	0.01	0.00	0.00	0.98	0.00	0.00
3165	0.00	0.00	0.01	0.00	0.13	0.00	0.00	0.85	0.00	0.01
3164	0.00	0.02	0.02	0.00	0.26	0.25	0.00	0.40	0.00	0.05
3163	0.00	0.03	0.01	0.00	0.29	0.30	0.00	0.33	0.00	0.04
3162	0.00	0.03	0.03	0.00	0.26	0.25	0.00	0.36	0.00	0.07
3161	0.00	0.02	0.01	0.00	0.53	0.17	0.00	0.25	0.00	0.02
3160	0.00	0.00	0.00	0.00	0.71	0.09	0.00	0.16	0.00	0.03
3159	0.00	0.00	0.01	0.00	0.70	0.16	0.00	0.11	0.00	0.02
3158	0.00	0.02	0.02	0.00	0.40	0.02	0.00	0.54	0.00	0.01